

Lucrarea nr. 4

Algoritmi de generare a primitivelor grafice de tip linie

1. OBIECTIVELE LUCRĂRII

Lucrarea își propune prezentarea modalităților de afișare a unei anumite figuri geometrice direct la echipamentele de ieșire. Pentru aceasta sunt prezentate proceduri care primesc ca date coordonatele de intrare și folosesc algoritmi pentru afișarea figurii geometrice la echipamentul de ieșire (monitor sau imprimantă).

2. BREVIAR TEORETIC

2.1 Primitive de ieșire

Un desen elaborat este constituit dintr-o combinație de figuri geometrice de bază:

- Puncte,
- Linii,
- Suprafețe poligonale,
- Liniile curbe,
- Șirurile de caractere.

Aceste componente de bază ale oricărui desen sunt numite *primitive de ieșire*. Pentru fiecare astfel de primitivă s-au definit algoritmi pentru afișare pe sisteme vectoriale sau raster. Algoritmii pot fi folosiți atât de procesorul de pe placa grafică (implementare hardware a algoritmilor) cât și în implementarea funcțiilor bibliotecilor grafice (OpenGL, Direct3D).

2.2 Algoritmi de desenare a liniei

Instrucțiunile de desenare a unor linii dintr-un program de aplicație corespund definirii coordonatelor extreme ale fiecărei linii urmând ca echipamentul de ieșire să unească prin linii drepte toate aceste coordonate.

La realizarea algoritmilor de desenare a liniei se pleacă de la ecuația matematică a unei linii care este:

$$y = m \cdot x + b, \quad x \in [x_1, x_2] \quad (1)$$

Dacă se cunosc coordonatele a două puncte aparținând liniei atunci pot fi determinate valorile m și b (Fig. 1).

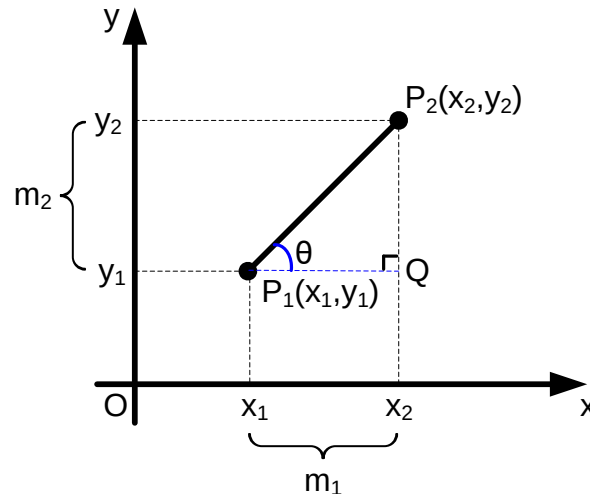


Figura 1

$$\operatorname{tg} \theta = m = \frac{P_2Q}{P_1Q} = \frac{y_2 - y_1}{x_2 - x_1} = \frac{m_2}{m_1} \quad (2)$$

$$b = y_1 - m \cdot x_1 \quad (3)$$

unde m reprezintă tangenta unghiului pe care îl face dreapta cu axa Ox sau panta, b reprezintă intersecția cu axa Oy a liniei, x reprezintă o variabilă independentă.

Algoritmii pentru afișarea unei linii se bazează pe relațiile anterioare și pe ecuația:

$$\Delta y = m \cdot \Delta x \quad (4)$$

2.2.1 Algoritmul DDA de trasare linie

Algoritmul DDA (**D**igital **D**ifferential **A**nalyzer) este un algoritm de calcul a poziției unui pixel de-a lungul unei linii. Se face mai departe o analiză în funcție de pantă, astfel:

- Dacă panta $m \leq 1$ ($\Delta x \geq \Delta y$), atunci și numărul punctelor reprezentate este mai mare dacă se consideră variația după x .

$$y_{i+1} = y_i + m \quad (5)$$

- Dacă $m > 1$ ($\Delta x < \Delta y$), atunci și numărul punctelor reprezentate este mai mare dacă se consideră variația după y . Se va inversa rolul jucat de coordonatele x și y în sensul că se determină valorile pentru coordonatele x după relația:

$$x_{i+1} = x_i + 1/m \quad (6)$$

Algoritmul DDA este o metodă mai rapidă pentru determinarea poziției pixelilor decât cea a folosirii ecuației (1).

2.2.2 Algoritmul Bresenham de trasare linie

Acest algoritm se bazează tot pe metoda incrementală (la fel ca și DDA), dar operează doar cu numere întregi.

În continuare prezentăm pașii algoritmului Bresenham pentru pantă pozitivă și mai mică decât unu.

- Se citesc coordonatele punctelor extreme (coordoanatele punctului din stânga (x_1, y_1) și coordonatele punctului din dreapta (x_2, y_2)).
- Se afișează punctul din stânga (x_1, y_1) .
- Se determină $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$ și parametrul $p_1 = 2 \cdot \Delta y - \Delta x$. Pentru $p_i < 0$ următorul punct afișat va fi (x_i+1, y_i) , iar altfel va fi (x_i+1, y_i+1) .
- Se continuă incrementarea unitară a coordonatelor x . Pentru poziția x_i+1 , coordonata selectată pentru verticală va fi y_i sau y_i+1 în funcție de semnul parametrului p_i : $p_i < 0$ sau $p_i \geq 0$. Modul de calcul al parametrului p depinde de valoarea anterioară a acestuia.
 - Dacă $p_i < 0$ atunci parametrul următor va fi: $p_{i+1} = p_i + 2 \cdot \Delta y$
 - Dacă $p_i \geq 0$ atunci parametrul următor va fi: $p_{i+1} = p_i + 2 \cdot (\Delta y - \Delta x)$
- Dacă $p_{i+1} < 0$ următoarea valoare pentru coordonata verticală va fi y_{i+1} , iar altfel va fi $y_{i+1}+1$.
- Se repetă operațiile de la punctul 4 până când coordonata x devine egală cu x_2 .

Algoritmul a fost descris doar pentru varianta $0 < m < 1$. Extinderea algoritmului pentru valori $m \geq 1$ se face prin inversarea rolului coordonatelor x și y . Algoritmul Bresenham pentru determinarea poziției pixelilor în cadrul unei linii este mult mai eficient decât DDA deoarece folosește doar operații cu întregi.

3. PROBLEME REZOLVATE

Problema 1

Algoritm pentru desenare linie ce are la bază ecuația (1).

Soluție

Programul în C este următorul:

```
#include <stdlib.h>
#include <conio.h>
#include <iostream.h>
#include <graphics.h>

void linie (int x1, int y1, int x2, int y2, int culoare)
{
    int dx = x2 - x1;
    int dy = y2 - y1;
    putpixel(x1, y1, culoare);

    // se construiește algoritmul de trasare tinand cont de ecuatia dreptei
    ....
}
```

```
}

void main()
{
    //autodetectare
    int gdriver = DETECT, gmode=EGAHI, errorcode;
    int x1, y1, x2, y2, culoare;

    //initializare variabile locale si globale
    initgraph(&gdriver, &gmode, "c:\\bc\\borlandc\\bgi");

    //rezultatele initializarii
    errorcode=graphresult();
    if(errorcode!=grOk)
    {
        cout<<"Graphics error: "<<grapherrormsg(errorcode)<<endl;
        cout<<"Apasa orice tasta pentru a se opri:";
        getch();
        exit(1);
    }

    //stabilire culoare
    culoare=14;
    x1 = 0;
    y1 = 0;
    x2 = getmaxx()/2;
    y2 = getmaxy()/2;
    linieSimpla(x1, y1, x2, y2, culoare);

    //stabilire culoare
    culoare=4;
    x1 = getmaxx()/2;
    y1 = getmaxy()/2;
    x2 = getmaxx();
    y2 = 0;
    linieSimpla(x1, y1, x2, y2, culoare);

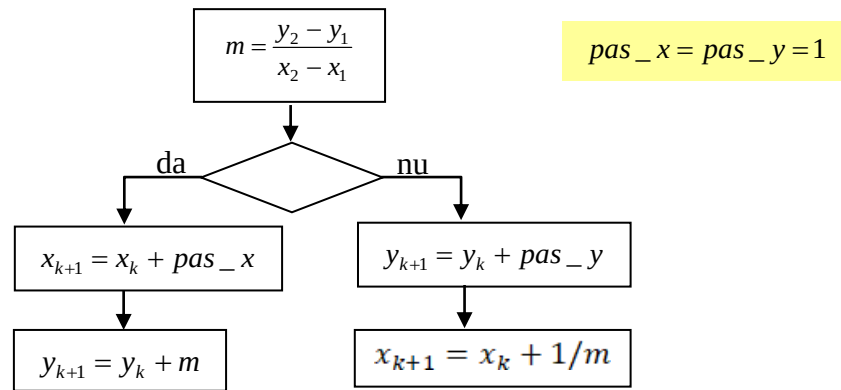
    getch();
    closegraph();
    return;
}
```

Problema 2

Realizați un program pentru implementarea algoritmului de trasare linie DDA.

Soluție

Pentru rezolvarea problemei se va ține cont de aspectele teoretice prezentate la punctul 2.2.1. O posibila secventa de organigrama este urmatoarea:



Algoritmul poate fi sintetizat într-o procedură care primește ca date de intrare coordonatele punctelor extreme (x_1, y_1) și (x_2, y_2) ale liniei de desenat.

Programul în C este următorul:

```

#include <stdlib.h>
#include <conio.h>
#include <iostream.h>
#include<graphics.h>
#include<math.h>

void dda(int x1, int y1, int x2, int y2, int color)
{
    int dx, dy;

    dx = x2-x1;
    dy = y2-y1;
    //se implementeaza algoritmul conform organigramei prezentata mai sus
    .....
}

void main()
{
    //autodetectare
    int gdriver = DETECT, gmode=EGAHI, errorcode;
    int x1, y1, x2, y2, color;

    //initializare variabile locale si globale
    initgraph(&gdriver, &gmode, "c:\\bc\\borlandc\\bgi");

    //rezultatele initializarii
    errorcode=graphresult();
    if(errorcode!=grOk)
    {
        cout<<"Graphics error: "<<grapherrormsg(errorcode)<<endl;
        cout<<"Apasa orice tasta pentru a se opri:";
        getch();
        exit(1);
    }

    //stabilire culoare
    color=14;
    x1 = 0;
    y1 = 0;
}

```

```
x2 = 10;
y2 = 5;
dda(x1, y1, x2, y2, color);

getch();
closegraph();
return;
}
```

Problema 3

Realizați un program pentru implementarea algoritmului de trasare linie Bresenham.

Soluție

Pentru rezolvarea problemei se va ține cont de pașii algoritmului Bresenham pentru pantă pozitivă și mai mică decât unu prezentați la punctul 2.2.2.

Programul în C este următorul:

```
void Bresenham(int x0, int y0, int x1, int y1, int culoare)
{
    int dy = y1 - y0;
    int dx = x1 - x0;

    // se construiește algoritmul de trasare ținând cont de aspectele
    prezentate la punctul 2.2.2

    ...
}
```

4. DESFĂȘURAREA LUCRĂRII

1. Se va citi breviarul teoretic. Se atrage atenția asupra faptului că aceste informații vor fi folosite și la desfășurarea altor lucrări de laborator.
2. Se vor studia exemplele prezentate pe parcursul acestui laborator urmărind efectul rulării acestora și buna lor execuție.
3. Se va extinde algoritmul Bresenham prezentat în aplicația rezolvată 3 astfel încât să permită și trasarea liniilor cu pantă mai mare sau egală cu 1.
4. Întocmirea unui referat cuprinzând rezultatele și concluziile desprinse din lucrare.



Bibliografie

- [1] **Anghelescu P.** – *Sisteme de prelucrare grafică, Note de curs*, 2020.
- [2] **Iglesias A.** – *Computer-Aided Geometric Design and Computer Graphics: Line Drawing Algorithms*, 2001.
- [3] **Bresenham, J.E.** - *Algorithm for computer control of a digital plotter*, *IBM Systems Journal*, January 1965, pag. 25-30.
- [4] <http://www.cs.unc.edu/~mcmillan/comp136/Lecture6/Lines.html>